



FACULDADE DE
CIÊNCIAS E TECNOLOGIA
UNIVERSIDADE DE
COIMBRA

Project 3: Privacy Through Homomorphic Encryption

Master's in Informatics Security

Academic Year: 2024/2025

Work done by:

João António Figueiredo de Macedo

2021220627

Lucas Miguel Simões Loberto

2021219107

Contents

- Introduction..... 2
- Scenario and Problem Definition 3
 - Scenario description 3
 - Problem to be solved 3
 - Dataset description 4
- Code Implementation..... 5
 - utils.py - CKKS context generation and management 5
 - data_holder.py: Encrypting salaries and displaying the results 7
 - data_analyzer.py - Calculating statistics in an encrypted context 8
- Results and Discussion 10
- Conclusion 11
- References 12

Introduction

The gender pay gap remains a significant challenge across the European Union, representing a substantial obstacle to achieving gender equality and sustainable economic growth. In order to address this issue, the European Parliament and the Council have approved Directive (EU) 2023/970. This directive requires all companies with 100 or more employees to disclose their gender pay gap. If the average difference in pay between women and men in any job category is equal to or greater than 5%, and there are no objective justifications, the company must conduct a joint pay assessment with employees' representatives.

However, the calculation and dissemination of these indicators poses a dilemma for organisations: the need for transparency clashes with the obligation to protect sensitive personal data, such as individual salaries. The solution lies in the application of homomorphic encryption (HE) techniques, which facilitate the execution of statistical operations directly on encrypted data, ensuring that the original values remain inaccessible to unauthorised individuals.

In this project, two roles are distinguished: Data Holder, responsible for encrypting the data set and, in the end, deciphering the results; and Data Analyzer, which receives only the encrypted data, performs the required calculations and returns them encrypted.

To illustrate the approach, a dataset containing 10,398 annual pay records is used, with the sex of each worker and their basic salary recorded. The objective is to calculate the absolute average difference in salary between men and women, as well as the percentage deviation from the average female salary. The confidentiality of individual values will be preserved at all times. These two indicators allow companies to check whether they exceed the 5 per cent threshold imposed by Directive 2023/970, triggering or not the obligation to carry out a joint salary assessment.

By combining legal requirements for salary transparency with state-of-the-art cryptographic techniques, the work illustrates how it is possible to fulfil new regulatory requirements without compromising workers' privacy, contributing to fairer and more informed salary decisions.

Scenario and Problem Definition

Scenario description

Directive (EU) 2023/970 requires all employers with 100 or more employees to publish gender pay gap indicators. In instances where the average pay gap in a professional category exceeds 5% and there are no objective justifications, the organisation is obligated to conduct a joint pay assessment with employee representatives. By integrating legal mandates for pay transparency with innovative encryption methodologies, the work demonstrates how to comply with new regulatory requirements without compromising employee privacy, thus facilitating fairer and more informed pay decisions.

In order to fulfil these obligations without exposing individual salaries - information considered sensitive by the GDPR - it is proposed to use homomorphic encryption.

The following is a summary of the process:

- Data Holder (RH) is responsible for encrypting the salary file and retaining the private key.
- Data Analyzer is responsible for receiving encrypted data, calculating the required metrics and returning them encrypted.
- Data Holder is responsible for decrypting the final results and checking the regulatory threshold.

This model successfully balances statistical transparency with the protection of individual confidentiality.

Problem to be solved

The challenge focuses on measuring the gender pay gap without disclosing individual salaries. Firstly, the objective is to establish the difference in average income between men and women, expressed in absolute terms. This discrepancy is then converted into a percentage of the average female salary, enabling immediate and standardised assessment of whether the disparity exceeds the regulatory threshold of 5%. In the event that the limit is exceeded, the company is legally obligated to undertake the joint pay assessment stipulated in the directive, involving workers' representatives in an examination of its remuneration policies. All of this must take place on encrypted data, ensuring that no specific salary becomes visible during the process.

Dataset description

As previously stated, the dataset contains 10,398 records. Each entry corresponds to a public employee and includes fields such as the employee's ID, annual base salary (in USD), and gender. The dataset was obtained from the U.S. government open data portal, specifically from the following link: <https://catalog.data.gov/dataset/employee-salaries-2024>. According to the portal, the dataset is publicly available, and it can be freely reused and shared.

The original file was provided in Excel (.xlsx) format. Following a thorough review, it was noted that the file in question contained several auxiliary columns, including "Department" and "Division". These were removed during the preprocessing stage, as they did not contribute any relevant information to the analysis. The relevant columns retained for the project were: Gender and Base Salary/Year (\$). The Employee ID column was also added, but it was not necessary for the statistical analysis and was therefore ignored to simplify processing.

Finally, it is important to note that salary data, even in anonymised form, is considered sensitive under many data protection frameworks, such as the GDPR. In this context, homomorphic encryption is employed in the subsequent phases of the project to ensure that any analysis conducted over this dataset respects the confidentiality of the individuals represented. This aligns with the project's broader goal of privacy-preserving data analysis.

Code Implementation

This chapter provides comprehensive descriptions of the three Python files that constitute the solution: `utils.py`, `data_holder.py` and `data_analyzer.py`. The architecture follows the standard data holder/data analyst model used in homomorphic encryption (HE) scenarios.

`utils.py` - CKKS context generation and management

This file is the core of the project's entire cryptographic infrastructure. Upon initialisation, the program's `main()` entry point verifies the presence of the `private_key` and `public_key`. In the absence of these files, the program will generate a CKKS context with a polynomial degree of 16,384 and a chain of coefficient sizes [60, 50, 40, 40, 40, 40, 60], adjusting the global precision with `global_scale = 2**40`. At this stage, the Galois and relinearisation keys are also generated. The complete context, containing the secret key, is serialised and stored in Base64 in the `private_key` file, while a public version is stored in `public_key`. This ensures that only the data holder retains the ability to decrypt.

```
def main():
    if os.path.exists("files/" + PRIVATE_KEY_FILENAME) and os.path.exists("files/" + PUBLIC_KEY_FILENAME):
        print("Private and public keys already exist.")
    else:
        print("Generating keys...")

        context = tenseal.context(
            tenseal.SCHEME_TYPE.CKKS,
            poly_modulus_degree=16384,
            coeff_mod_bit_sizes=[60, 50, 40, 40, 40, 40, 60]
        )

        context.generate_galois_keys()
        context.generate_relin_keys()
        context.global_scale = 2**40

        secret_context = context.serialize(save_secret_key=True)
        write_data(secret_context, PRIVATE_KEY_FILENAME)
        print("Private key created and saved.")

        context.make_context_public()
        public_context = context.serialize()
        write_data(public_context, PUBLIC_KEY_FILENAME)
        print("Public key created and saved.")
```

The persistence of these objects is ensured by the `write_data` and `read_data` functions: the former receives any sequence of bytes (contexts, encrypted vectors, etc.), encodes it in Base64 and saves it in the `files/` folder, creating it if necessary; the latter performs the reverse operation, returning the original bytes ready to be consumed by TenSEAL. In order to recreate the context, there are two functions: `load_context_secret()` (for the data holder) and `load_context_public()` (for the data analyst).

```

def write_data(data: bytes, filename: str):
    data = base64.b64encode(data)

    filepath = "files/" + filename
    if not os.path.exists("files"):
        os.makedirs("files")

    with open(filepath, "wb") as file:
        file.write(data)

def read_data(filename: str) -> bytes:
    filepath = "files/" + filename
    if not os.path.exists("files"):
        os.makedirs("files")

    with open(filepath, "rb") as file:
        data = file.read()

    return base64.b64decode(data)

def load_context_secret():
    return tenseal.context_from(read_data(PRIVATE_KEY_FILENAME))

def load_context_public():
    return tenseal.context_from(read_data(PUBLIC_KEY_FILENAME))

```

Finally, `pack_cts` facilitates the exchange of multiple ciphertexts within a single payload. For each encrypted vector, its serialisation is obtained, preceded by four bytes indicating its length in 'little-endian'. The result is a compact sequence that can be written to a file and later separated deterministically during the decryption phase.

```

def pack_cts(*cts):
    chunks = []
    for ct in cts:
        ct_serialized = ct.serialize()
        chunks.append(len(ct_serialized).to_bytes(4, "little"))
        chunks.append(ct_serialized)

    return b"".join(chunks)

```

The `utils.py` file provides a cohesive set of utilities that streamlines the entire cycle of key generation, secure (de)serialisation and data packaging. This enables the remaining modules to focus on the actual ciphering and statistical analysis operations.

data_holder.py: Encrypting salaries and displaying the results

The data_holder.py file is run on the side that has the data in the clear, and it brings together, in a single script, all the logic for pre-processing, encrypting and then decrypting the aggregated results. When invoked, it accepts the --mode command line argument, with two options: encrypt (the default value) or decrypt. This interface enables each phase to be tested in isolation, thereby accelerating the development and demonstration cycle.

```
if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--mode",
                        choices=["encrypt", "decrypt"],
                        default="encrypt",
                        help="Executar apenas encrypt ou apenas decrypt.")
    args = parser.parse_args()

    if args.mode in ("encrypt"):
        male, female = excel_to_salary_lists("dataset.xlsx")
        print("Encrypting male data...")
        encrypt(male, "encrypted_male_data")

        print("Encrypting female data...")
        encrypt(female, "encrypted_female_data")

    elif args.mode in ("decrypt"):
        print("Decrypting results...")
        decrypt("statistics")
```

In encrypt mode, the excel_to_salary_lists() function loads the dataset.xlsx file with Pandas, selects only the Gender and Base Salary/Year (\$) columns and separates it into two float lists: one for men (M) and one for women (F). Each list is then passed to encrypt(dataset, filename), which reconstructs the CKKS context with a secret key (load_context_secret()), generates a TenSEAL vector and writes the serialisation in Base64 to the encrypted_male_data or encrypted_female_data via write_data().

```
def encrypt(dataset, filename):
    context = load_context_secret()
    encrypted_data = tenseal.ckks_vector(context, dataset)
    write_data(encrypted_data.serialize(), filename)

def excel_to_salary_lists(filename):
    df = pd.read_excel(filename)

    male = df.loc[df['Gender'] == 'M', 'Base Salary/Year ($)'].astype(float).tolist()
    female = df.loc[df['Gender'] == 'F', 'Base Salary/Year ($)'].astype(float).tolist()

    return male, female
```


In decrypt mode, the script reads the files/statistics payload, which is produced by the data analyser. This payload contains four ciphertexts arranged sequentially; the `decrypt(filename)` function runs through them, retrieves the size of each block (four little-endian bytes per vector), reconstructs the CKKS object with `ckks_vector_from`, decrypts each value and displays it on screen to two decimal places: male average, female average, absolute difference and gender pay gap in percentage.

```
def decrypt(filename):
    context = load_context_secret()
    enc_proto = read_data(filename)

    offset = 0
    vals = []
    for _ in range(4):
        length = int.from_bytes(enc_proto[offset:offset + 4], "little")
        offset += 4
        ct_serialized = enc_proto[offset:offset + length]
        offset += length
        ct = tenseal.ckks_vector_from(context, ct_serialized)
        vals.append(ct.decrypt()[0])

    mean_m, mean_f, gap, gap_pct = vals
    print(f"Média M      = {mean_m:.2f}")
    print(f"Média F      = {mean_f:.2f}")
    print(f"Gap           = {gap:.2f}")
    print(f"Percentage gap = {gap_pct:.2f} %")
```

In this way, the data holder only obtains global statistics, never individual salary figures, while preserving the confidentiality required by the GDPR and Directive (EU) 2023/970.

data_analyzer.py - Calculating statistics in an encrypted context

The `data_analyzer.py` file is designed to process encrypted data. From the outset, the script establishes the CKKS context using `load_context_public()`. This process instantiates the same schema utilised by the Data Holder. The next step is to read the `encrypted_male_data` and `encrypted_female_data` files using the `read_data` function and then convert them back into CKKS vectors via `tenseal.ckks_vector_from`. This process prepares the data for homomorphic processing.

The calculation of the average exploits the fact that CKKS supports sum and multiplication. Each vector runs `.sum()`, which calculates the sum of the salaries. This is then multiplied by the scalar $1/|\text{vector}|$ (the inverse of the vector size). Since the scalar is in the clear, the result remains encrypted. The mean values for males

and females are then stored in the 'average_male_salary' and 'average_female_salary' fields, respectively.

```
context = load_context_public()

male_salary_proto = read_data("encrypted_male_data")
male_salary_encrypted = tenseal.ckks_vector_from(context, male_salary_proto)
male_salary_sum_encrypted = male_salary_encrypted.sum()
n_male = 1 / male_salary_encrypted.size()

female_salary_proto = read_data("encrypted_female_data")
female_salary_encrypted = tenseal.ckks_vector_from(context, female_salary_proto)
female_salary_sum_encrypted = female_salary_encrypted.sum()
n_female = 1 / female_salary_encrypted.size()

average_male_salary = male_salary_sum_encrypted * n_male
average_female_salary = female_salary_sum_encrypted * n_female
```

The absolute gender pay gap is simply the difference between these two averages ($\text{gap} = \text{average_male_salary} - \text{average_female_salary}$). To calculate the percentage value, the gap must be divided by the female average. However, direct division is not possible in HE; therefore, the code applies a single Newton-Raphson iteration, starting with the initial guess of $1/100,000$, to produce a sufficiently accurate approximation of the inverse of the average female salary. Multiplying this inverse by the gap and by 100 gives gap_percentage , all still encrypted.

```
gap = average_male_salary - average_female_salary

inv_guess = 1.0 / 100_000.0
inv_average_female_salary = inv_guess * (2.0 - average_female_salary * inv_guess)
gap_percentage = gap * inv_average_female_salary * 100.0
```

Finally, the four results – average male, average female, absolute gap and percentage gap – are packaged into a single payload using the `pack_cts` function. This gives each vector its size in four little-endian bytes, and the payload is written to 'files/statistics' with the `write_data` function. This format facilitates successive sequential decryption by the Data Holder, ensuring that only aggregated indicators cross the border between the two entities.

```
print("Writing statistics to file...")
payload = pack_cts(average_male_salary, average_female_salary, gap, gap_percentage)
write_data(payload, "statistics")
```

Results and Discussion

In order to run the complete pipeline, it is first necessary to run 'python utils.py' at the root of the project. This will generate the CKKS keys and save the 'files/private_key' and 'files/public_key' files. Then, run 'python data_holder.py --mode encrypt' to read the dataset.xlsx, filter the gender and salary columns, separate men and women and save the encrypted vectors in 'files/encrypted_male_data' and 'files/encrypted_female_data', then run 'python data_analyzer.py'. In order to load these files with the public context, the following steps should be taken: first, calculate sums, averages, absolute differences and percentage gaps in ciphertext; then, package the four encrypted results in 'files/statistics'; finally, call 'python data_holder.py --mode decrypt' to read this payload, decrypt each metric and print the results on screen.

```
python utils.py
Generating keys...
Private key created and saved.
Public key created and saved.

python data_holder.py --mode encrypt
Encrypting male data...
Encrypting female data...

python data_analyzer.py
Calculating statistics...
Writing statistics to file...

python data_holder.py --mode decrypt
Decrypting results...
Média M          = 99661.24
Média F          = 96655.32
Gap              = 3005.92
Percentage gap   = 3.11 %
```

The following summary outlines all the indicators:

<i>Metric</i>	<i>Valor</i>
Average male salaries	99 661,24 USD
Average female salaries	96 655,32 USD
Absolute gap	3 005,92 USD
Percentage gap	3,11 %

In summary, the average annual salary for women is \$3,005.92 less than that of men, representing a 3.11% shortfall compared to the male mean salary. Therefore, the gap is situated beneath the 5% transparency threshold stipulated by Directive (EU) 2023/970.

Conclusion

This project successfully demonstrated how homomorphic encryption can be leveraged to measure sensitive metrics, such as the gender pay gap, while preserving individual salary confidentiality. Utilising a dataset comprising 10,398 anonymised employee salary records, the workflow securely calculated the average male and female salaries, revealing an absolute difference of 3,005.92 USD, corresponding to a 3.11% pay gap.

The obtained value for the gap is comfortably below the 5% threshold established by Directive (EU) 2023/970, meaning the analysed organisation is not legally required to conduct a joint pay assessment at this stage. However, the identified discrepancy, although below the regulatory threshold, still indicates the presence of salary inequalities worthy of further internal examination.

From a privacy perspective, the project is fully compliant with GDPR guidelines. Throughout the process, salary information remained encrypted, never leaving the data-holder's environment in plaintext. The use of CKKS encryption in this pipeline proved robust and computationally efficient, validating the practical feasibility of performing secure statistical analysis at scale.

References

- [1] Sefik Serengil, “A Step by Step Fully Homomorphic Encryption Example with TenSEAL in Python - Sefik Ilkin Serengil,” *Sefik Ilkin Serengil*, Apr. 10, 2023. <https://sefiks.com/2023/04/10/a-step-by-step-fully-homomorphic-encryption-example-with-tenseal-in-python/> (accessed May 20, 2025).
- [2] “TenSEAL,” GitHub, Jul. 24, 2023. <https://github.com/OpenMined/TenSEAL>
- [3] “tenseal,” PyPI, Jul. 06, 2020. <https://pypi.org/project/tenseal/0.1.0a0/> (accessed May 20, 2025).
- [4] OpenMined, “OpenMined/TenSEAL,” GitHub, 2025. <https://github.com/OpenMined/TenSEAL/issues/> (accessed May 20, 2025).
- [5] OpenMined, “TenSEAL/tutorials at main · OpenMined/TenSEAL,” GitHub, 2020. <https://github.com/OpenMined/TenSEAL/tree/main/tutorials> (accessed May 20, 2025).
- [6] Slides das aulas